



Mamba-attention: A self-supervised framework for efficient remaining useful life prediction

Jihun Han¹, Yejin Kwon¹, Hyunsoo Yoon^{1*}

Department of Industrial Engineering, Yonsei University, 50 Yonsei-ro, Seodaemun-gu, Seoul, 03722, Republic of Korea

ARTICLE INFO

Dataset link: <https://data.nasa.gov/dataset/cm-apss-jet-engine-simulated-data>, <https://calce.umd.edu/battery-data>

Keywords:

Remaining useful life
State space model
Mamba
Attention
Self-supervised learning

ABSTRACT

Accurate remaining useful life (RUL) prediction provides significant economic, technical, and safety benefits by forecasting the lifespans of industrial equipment. Current state-of-the-art (SOTA) RUL prediction models predominantly rely on recurrent neural networks (RNNs) and attention-based models. While RNNs are adept at capturing temporal dependencies, they struggle with issues such as vanishing gradients and exhibit inefficiency in processing long sequences. Meanwhile, attention-based models improve the capture of global context but are limited by their high computational complexity. The proposed Mamba-Attention model addresses these limitations by integrating a selective state space model with attention mechanisms, achieving SOTA performance with faster training times on C-MAPSS dataset. Industrial equipment is frequently replaced prematurely, and obtaining labeled data is challenging due to cost and time constraints. To overcome these difficulties, this research introduces a novel framework that leverages unlabeled data via self-supervised learning approaches, including temporal loss, N-tuplet loss, and pseudo-label loss. By pre-training with self-supervised losses and fine-tuning on minimal labeled data in few-shot scenarios, the proposed approach remains effective in producing accurate RUL predictions, even under limited supervision, as demonstrated on the C-MAPSS and CALCE datasets. This reduces the reliance on costly labeled datasets and offers practical solutions for industrial applications.

1. Introduction

Prognostics and health management (PHM) is essential for ensuring the reliability of safety-critical industrial systems, such as nuclear plants and oil facilities [1]. A key aspect of PHM is remaining useful life (RUL) prediction, which provides timely maintenance alerts, reducing costs and preventing unexpected failures [2,3]. With the advent of industry digitalization and IoT, data-rich environments have enabled sophisticated data-driven methods for RUL estimation [2].

RUL prediction methods generally fall into physical model-based, knowledge-based, and data-driven approaches [4]. Physical models simulate equipment degradation but struggle with complex scenarios [5]. Knowledge-based methods depend heavily on domain expert experience and lack generalizability [6]. Data-driven methods, however, offer superior flexibility and generalization by directly learning degradation patterns from sensor data using machine learning [7,8].

Deep learning models, including multi-layer perceptrons (MLP) [9], recurrent neural networks (RNN) [10–12], convolutional neural networks (CNN) [9], and attention-based models [13,14], have been developed for improved RUL prediction. Nevertheless, RNNs face vanishing gradient issues, and attention-based models suffer from high

computational costs and may overlook temporal details. To address these challenges, our research introduces the Mamba architecture [15], which efficiently models long-range dependencies without the gradient issues of RNNs or scalability challenges of Transformers.

Moreover, supervised deep learning models require extensive labeled data, often impractical in industrial settings due to preventive maintenance practices. To tackle limited labeled data availability, self-supervised learning, successful in NLP and image domains, has potential for RUL tasks but remains underexplored due to the temporal complexity of sensor data. Traditional self-supervised techniques, such as masking, can disrupt temporal continuity, impairing feature extraction.

This study proposes a novel framework that integrates three self-supervised losses — temporal loss, N-tuplet loss, and pseudo-label loss — to address the challenge of RUL prediction in limited-label scenarios. The encoder is first pre-trained using only unlabeled sensor data, followed by supervised fine-tuning with minimal labeled data, demonstrating strong predictive performance even in one-shot learning setups.

* Corresponding author.

E-mail address: hs.yoon@yonsei.ac.kr (H. Yoon).

¹ Jihun Han and Yejin Kwon contributed equally to this work as co-first authors.

We adopt the standard zero-life labeling convention used in prior RUL prediction literature [10], where the failure point is labeled as $RUL = 0$. In our framework, “one-shot” refers to using a single labeled sample (i.e., one engine for C-MAPSS or one battery for CALCE) to train the final prediction head, while the encoder remains frozen. This approach enables the model to generalize RUL predictions across different units using extremely limited supervision, making it well-suited for real-world industrial settings where labeled degradation trajectories are often scarce. Furthermore, the framework naturally extends to few-shot scenarios, maintaining label efficiency while improving robustness.

Key contributions of this work include:

- Achieving state-of-the-art (SOTA) performance in RUL prediction by addressing the limitations of RNNs and attention-based models through the use of the MambaAtt architecture.
- Applying self-supervised learning to RUL tasks using a combination of temporal pattern recognition (temporal loss), contrastive learning (N-tuplet loss), and pseudo-labeling (pseudo-label Loss), thereby improving model robustness even with limited labeled data.
- Demonstrating the effectiveness of few-shot learning in RUL prediction, showing that the proposed method can achieve competitive results comparable to models that do not use self-supervised learning losses with minimal labeled data.

2. Related works

2.1. RNN-based approaches

Recurrent neural networks (RNNs), including variants like LSTM [16] and GRU [17], excel at capturing temporal dependencies in sequential data. GRU, in particular, has shown effectiveness in RUL prediction by reducing computational complexity and training time [18]. Bi-directional GRUs (Bi-GRU) further enhance temporal dependency modeling, addressing limitations of unidirectional GRUs [19]. However, RNN models generally struggle with long-term dependencies, motivating research into attention-based approaches.

2.2. Attention-based approaches

The Transformer [20] significantly advanced sequence modeling by effectively capturing long-range dependencies. Mo et al. (2021) applied a Transformer with gated convolutional units for industrial RUL prediction [14]. However, Transformers face quadratic computational complexity and lack sequential structural modeling. To mitigate these issues, Chang et al. (2022) introduced a temporal flow Transformer [21], while Shi et al. (2021) combined LSTM and attention mechanisms to better capture sequential degradation patterns [22].

2.3. Mamba architecture

Mamba is a selective state space model that dynamically controls information flow across sequences, enabling efficient handling of long-range dependencies while discarding irrelevant signals [15]. This design overcomes key limitations of RNNs (e.g., vanishing gradients, limited memory) and Transformers (e.g., quadratic complexity in sequence length), achieving improved scalability and performance. Originally developed for tasks such as language modeling, vision, and time series analysis [23], Mamba has recently been applied to battery health estimation, including SOH, SOC, and RUL prediction for lithium-ion batteries [24,25], demonstrating its versatility in prognostics. To the best of our knowledge, this work presents the first application of a hybrid Mamba-attention architecture for RUL prediction on the C-MAPSS dataset.

2.4. Self-supervised learning in RUL prediction

Self-supervised learning leverages the data itself to generate supervisory signals, allowing models to learn valuable representations without requiring labeled data. This approach is particularly beneficial in the estimation of RUL, where labeled data can be scarce. Self-supervised learning methods are typically divided into contrastive and generative approaches [26]. Contrastive methods focus on training an encoder to produce explicit vector representations, measuring the similarity between these representations to learn meaningful features. Recent studies have applied contrastive methods to RUL prediction. Zhu et al. (2024) introduce the Contrastive Bi-LSTM enabled health representation learning (CBHRL) framework for RUL prediction, utilizing supervised contrastive learning to achieve superior performance over existing methods [27]. However, this approach is not fully self-supervised, indicating potential for future research in semi-supervised or fully self-supervised learning paradigms. Similarly, Yu et al. (2023) proposes a self-supervised learning approach built upon a variational autoencoder (VAE) for feature extraction in RUL prediction, clustering the extracted features to mitigate the negative effect of varying working conditions and fault modes on RUL prediction accuracy [28]. However, this method relies on self-supervised learning mainly for feature extraction, with the final task of RUL prediction performed using supervised learning on clustered data. In contrast, our approach aims to achieve one-shot or few-shot RUL predictions using minimal labeled data. Wang et al. (2023) propose a self-supervised health representation decomposition learning (SHRDL) framework, which employs data augmentation and a contrastive loss modified by cycle information, based on a modified Info-NCE, to extract robust health representations [29]. Although this study improves RUL prediction, it still depends on supervised learning after downstream tasks after fine-tuning, and it does not experiment with one-shot or few-shot learning scenarios, an area where our study focuses on enabling efficient RUL prediction with limited labeled data.

3. Methods

3.1. Overall framework

The proposed framework comprises three main components: a Mamba encoder, a Transformer decoder, and a self-supervised training strategy. The overall architecture is illustrated in Fig. 1, while the training procedure is detailed in Fig. 3. The complete process is also summarized in Algorithm 1.

The first part is the Mamba block, which extracts features using Mamba, integrating the advantages of both RNNs and Transformers. Mamba effectively captures global context and handles long-term dependencies better than RNNs, making it advantageous for processing complex sequence data. Additionally, it maximizes computational efficiency by reducing the quadratic computational complexity of Transformers to linear time complexity.

The second part uses the Transformer block to make accurate predictions from the feature embeddings. The Transformer block leverages multi-head attention to focus on various parts of the input sequence, effectively managing long-term dependencies and enabling parallel processing, which makes it highly efficient for sequence generation tasks.

The final component is the self-supervised training strategy. Unlike existing RUL self-supervised learning methods that rely solely on contrastive learning, our work incorporates a novel loss function designed to learn sequential dependencies, thereby enabling the extraction of more temporal and contextually relevant features.

3.2. Mamba block

Mamba addresses the limitations of both RNNs and Transformer models, offering a scalable and efficient alternative that overcomes the computational and memory inefficiencies often encountered in other models.

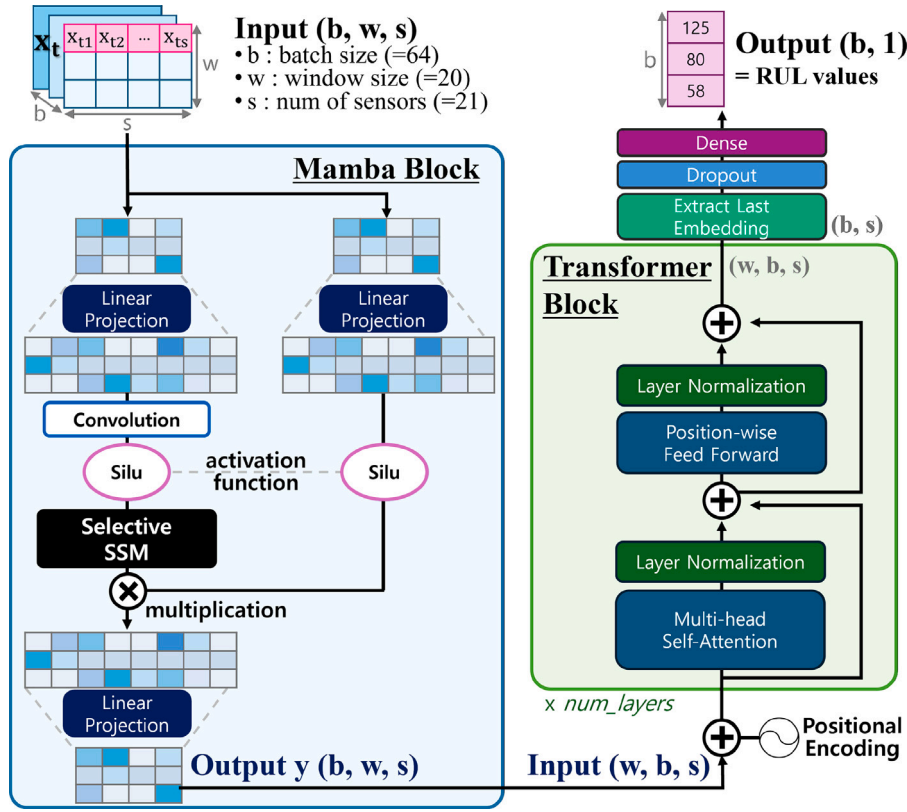


Fig. 1. Overall architecture of MambAtt.

3.2.1. State space model

To efficiently and effectively extract representations from sensors, MambAtt employs the Mamba model as an encoder. The Mamba model is constructed based on the fundamental concept known as the state-space model (SSM). The SSM is a mathematical framework designed to represent and analyze systems that evolve over time. It maps an input sequence $x(t) \in \mathbb{R}^n$ to an output sequence $y(t) \in \mathbb{R}^m$ via an internal state $h(t) \in \mathbb{R}^d$, defined by the following equations:

$$\begin{aligned} h'(t) &= Ah(t) + Bx(t) \\ y(t) &= Ch(t) \end{aligned} \quad (1)$$

where $A \in \mathbb{R}^{d \times d}$ is the state transition matrix, $B \in \mathbb{R}^{d \times n}$ is the input-to-state matrix, and $C \in \mathbb{R}^{m \times d}$ is the state-to-output matrix. The term $h'(t)$ denotes the time derivative of the state vector $h(t)$. Since discretization enables efficient computation by linear recursive method [30], the model was discretized by converting the continuous form (A, B, C) to discrete form $(\bar{A}, \bar{B}, \bar{C})$ by applying zero-order hold [31] with time sampling at intervals of Δ :

$$\begin{aligned} h_{k+1} &= h_k + \bar{B}x_k \\ y_k &= \bar{C}h_k \\ \bar{A} &= \exp(\Delta A,) \\ \bar{B} &= (\Delta A)^{-1} (\exp(\Delta A) - I) \cdot \Delta B, \end{aligned} \quad (2)$$

The discretized state equation, $h_{k+1} = \bar{A}h_k + \bar{B}x_k$, illustrates the SSM's recurrent structure, where each state h_{k+1} depends linearly on h_k and x_k . The output $y_k = \bar{C}h_k$ is a linear transformation of the state. Unlike RNNs, this linear model structure enables efficient parallel computation, resulting in faster processing while still capturing sequential dependencies.

The structured state space (S4) model [32] builds on SSM by integrating the HiPPO [33] to tackle long-range dependency challenges. Since $\bar{A}$ captures only the immediate past state, HiPPO effectively compresses continuous representations into finite dimensions, enabling S4 to model extended dependencies efficiently.

3.2.2. Selective state space model and mamba block

As shown in Fig. 2, the selective State Space Model enhances sequence modeling by dynamically adapting to input content, addressing the limitations of traditional SSMs that use static, time-invariant parameters A , B , and C . By making the matrices B , C , and even the step size Δ input-dependent, the model can selectively retain and prioritize relevant information based on context. To further enhance efficiency, Mamba integrates a hardware-optimized linear processing algorithm.

In Mamba, the input and output maintain the same shape, denoted $\mathbb{R}^{B \times W \times S} \rightarrow \mathbb{R}^{B \times W \times S}$, where B represents the batch size, W the window size, and S the number of sensors. The input first undergoes linear projections to generate vectors x and z with size E . A one-dimensional convolution, followed by a SiLU activation [34], is applied to x , resulting in x' . The matrix A is initialized as a structured state space, while B and C are derived from x' through linear transformations. The size of the step Δ is calculated using a Softplus function applied to a combination of parameters and a transmitted transformation of x' , making B , C , and Δ adaptable to the input sequence. Although A is not directly data dependent, the transformation $\bar{A} = \exp(\Delta A)$ introduces a degree of adaptability. This design enables Mamba to dynamically adjust its behavior based on the sequence context, establishing it as an input-dependent selective model. Although the original Mamba architecture was designed for sequence modeling tasks with per-timestep outputs, we adapt it for scalar regression by introducing additional modules tailored for RUL prediction, as described in Section 3.3. This includes a Transformer-based head that enables both temporal aggregation and selective attention over the sequence dimension.

3.3. Transformer block

To predict the final scalar RUL values from the embeddings obtained through the Mamba Encoder, we implement an additional Transformer block [20] that serves two purposes: (1) aggregating temporal features across the sequence dimension, and (2) selectively attending to the

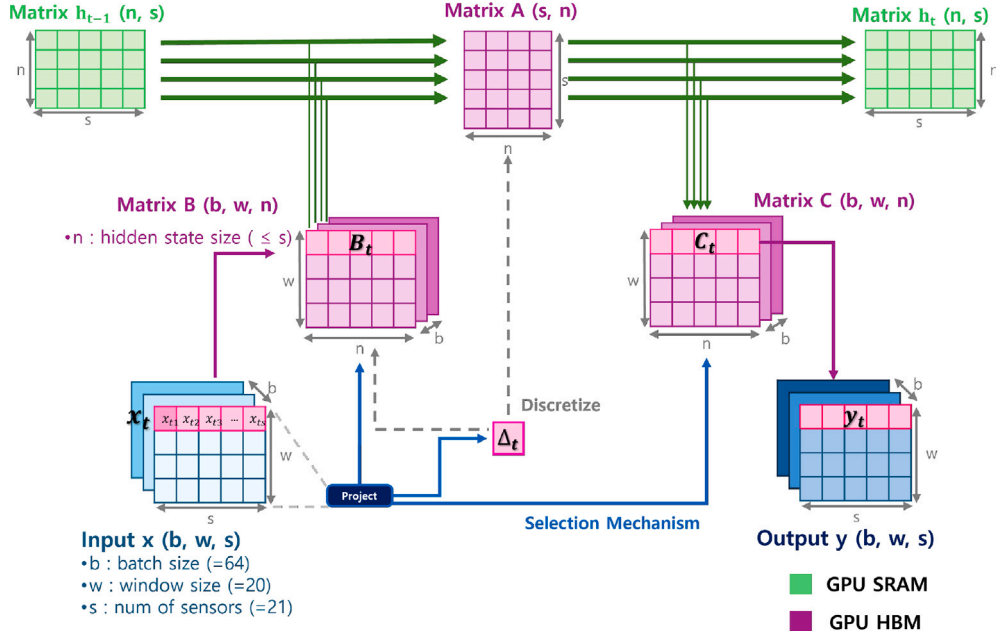


Fig. 2. Selective state space model.

Algorithm 1 Process of MambAtt

Input: $X : (B, W, S)$ ▷ Input data
Output: $Y_{\text{pred}} : (B, 1)$ ▷ Predicted RUL values

1: **Mamba Block**

2: $x : (B, W, E) \leftarrow \text{Linear}_x(X)$ ▷ Linear projection
3: $z : (B, W, E) \leftarrow \text{Linear}_z(X)$
4: $x' : (B, W, E) \leftarrow \text{SiLU}(\text{Conv1D}(x))$
5: $A : (W, N) \leftarrow \text{Parameter}$ ▷ Structured state matrix
6: $B : (B, W, N) \leftarrow \text{Linear}_B(x')$
7: $C : (B, W, N) \leftarrow \text{Linear}_C(x')$
8: $\Delta : (B, W, S) \leftarrow \text{Softplus}(\text{Parameter} + \text{Broadcast}(\text{Linear}(x')))$
9: $\bar{A}, \bar{B} : (B, W, S, N) \leftarrow \text{discretize}(\Delta, A, B)$ ▷ Input-dependent parameters and discretization
10: $y : (B, W, E) \leftarrow \text{SelectiveSSM}(\bar{A}, \bar{B}, C)(x')$
11: $y' : (B, W, E) \leftarrow y \otimes \text{SiLU}(z)$
12: $Z : (B, W, S) \leftarrow \text{Linear}(y')$

13:

14: $Z_{\text{trans}} : (W, B, S) \leftarrow \text{Transpose}(Z)$ ▷ Dimension adjustment
15: $PE : (W, 1, S) \leftarrow \text{PositionalEncoding}(W, S)$ ▷ Generate positional encoding

16: $Z_{\text{pos}} : (W, B, S) \leftarrow Z_{\text{trans}} + PE$ ▷ Add positional encoding
17: $Z_{\text{input}} \leftarrow Z_{\text{pos}}$ ▷ Initialize input for the first layer

18: **Transformer Block**

19: **for** $i = 1$ **to** L **do** ▷ Iterate through Transformer layers
20: $Z_{\text{norm1}} : (W, B, S) \leftarrow \text{LayerNorm}(Z_{\text{input}})$ ▷ Layer normalization
21: $Z_{\text{attn}} : (W, B, S) \leftarrow \text{MultiHeadAttention}(Z_{\text{norm1}})$ ▷ Multi-head self-attention
22: $Z_{\text{res1}} : (W, B, S) \leftarrow Z_{\text{input}} + Z_{\text{attn}}$ ▷ Residual connection
23: $Z_{\text{norm2}} : (W, B, S) \leftarrow \text{LayerNorm}(Z_{\text{res1}})$ ▷ Layer normalization
24: $Z_{\text{ffn}} : (W, B, S) \leftarrow \text{FeedForward}(Z_{\text{norm2}})$ ▷ Feed-forward network
25: $Z_{\text{res2}} : (W, B, S) \leftarrow Z_{\text{res1}} + Z_{\text{ffn}}$ ▷ Residual connection
26: $Z_{\text{input}} \leftarrow Z_{\text{res2}}$ ▷ Set input for the next layer
27: **end for**

28:

29: $Y_{\text{last}} : (B, S) \leftarrow \text{ExtractLast}(Z_{\text{input}})$ ▷ Extract last time-step embedding from the final layer output
30: $Y_{\text{drop}} : (B, S) \leftarrow \text{Dropout}(Y_{\text{last}})$ ▷ Apply dropout
31: $Y_{\text{pred}} : (B, 1) \leftarrow \text{Dense}(Y_{\text{drop}})$ ▷ Final prediction output

most informative time steps. This compensates for the lack of temporal pooling in the original Mamba model and enhances the interpretability and expressiveness of the final prediction layer. The output $\mathbb{R}^{B \times W \times S}$ of Mamba block is transposed to $\mathbb{R}^{W \times B \times S}$ to align with the Transformer's input requirements. Positional encoding is added to these embeddings to inject sequence order information:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right),$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \quad (3)$$

where pos is the position index, i is the dimension index, and d_{model} is set to 21, corresponding to the number of sensor types in the C-MAPSS data. The positional encodings are added to the input embeddings:

$$\mathbf{X}_{\text{pos}, b, i}^{\text{PE}} = \mathbf{X}_{\text{pos}, b, i} + \text{PE}(\text{pos}, i). \quad (4)$$

The Transformer block comprises multi-head self-attention layers, position-wise feed-forward networks, residual connections, and layer normalization. The multi-head attention mechanism allows the model to capture dependencies across different positions within a sequence. Queries Q , keys K , and values V are computed as:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (5)$$

where $X \in \mathbb{R}^{l \times d_{\text{model}}}$ is the input, and $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned projection matrices.

The attention scores are calculated using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (6)$$

For multi-head attention, the outputs from multiple attention heads are concatenated:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (7)$$

where each head is $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, and W^O is the output projection matrix.

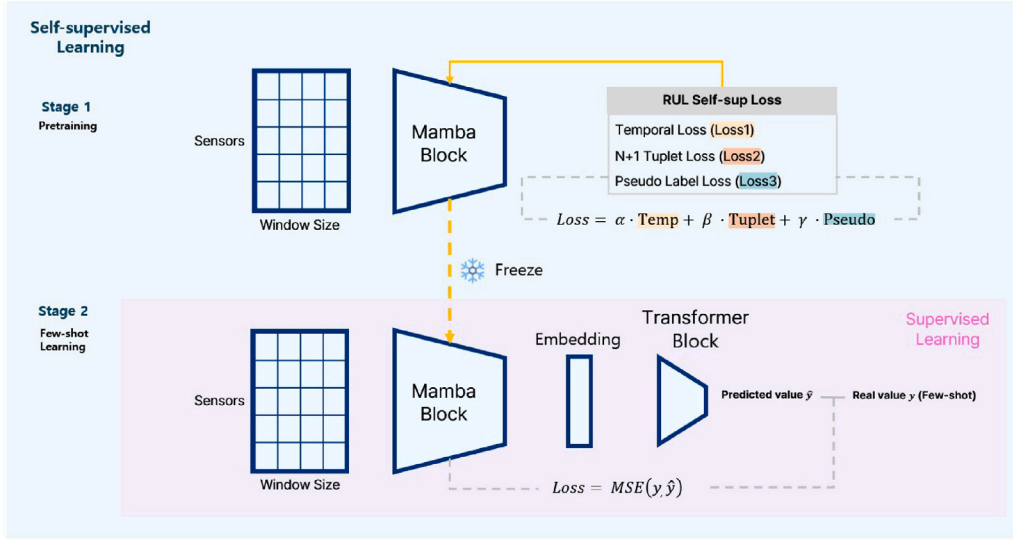


Fig. 3. Self-supervised learning framework.

Residual connections and layer normalization are applied to enhance training stability:

$$\text{LayerNorm}(\mathbf{X} + \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V})), \quad (8)$$

$$\text{LayerNorm}(\mathbf{X} + \text{FeedForward}(\mathbf{X})).$$

The feed-forward network consists of two linear transformations with a ReLU activation:

$$\text{FeedForward}(\mathbf{X}) = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (9)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

The output of the Transformer block maintains the shape $\mathbb{R}^{W \times B \times S}$. We extract the last time-step embedding to obtain $\mathbb{R}^{B \times S}$, which is then passed through a dense layer to produce the final RUL predictions with dimensions $\mathbb{R}^{B \times 1}$. By integrating the Transformer block, our model effectively captures complex temporal dependencies, decoding the feature representations obtained from the Mamba Encoder and allowing for more accurate RUL predictions.

3.4. Self-supervised learning

$$L_{\text{total}} = L_{\text{Temporal}} + L_{N\text{-tuple}} + L_{\text{Pseudo-label}} \quad (10)$$

As mentioned in Section 3.2.2, the Mamba model used as the encoder outputs the same shape as the input, $\mathbb{R}^{B \times W \times S} \rightarrow \mathbb{R}^{B \times W \times S}$. To extract high-quality features, as illustrated in Fig. 3, we employ the following three loss functions: temporal loss, N-tuplet loss, and pseudo-label loss. The first two losses are designed to incorporate sequential patterns into the embeddings and do not involve any form of label information, directly or indirectly. The last loss assigns pseudo-labels to guide the health state of each data segment within the sequence.

3.4.1. Temporal ordering loss

In the context of RUL data, values evolve over time, reflecting progressive equipment degradation. To capture these temporal dynamics, we employ the temporal ordering loss (temporal loss), inspired by the method in [35], which encourages the model to distinguish between correct and incorrect sequential orders. In each batch, we construct a total of eight triplets: six that are correctly ordered and two that are intentionally disordered. These triplets are formed by sampling three embeddings from within a temporal window to reflect both short-range and long-range dependencies. For example, temporal triplets such as $[i-1, i, i+1]$ and $[i-2, i, i+3]$ preserve the correct temporal order and are labeled as ordered, while permutations such as $[i+2, i, i+1]$

violate the natural sequence and are labeled as disordered. This process is illustrated in Fig. 4(a), which provides a visual overview of how ordered and disordered triplets are constructed. This results in a triplet tensor of shape $\mathbb{R}^{8 \times 3 \times L \times D}$ and corresponding binary labels of shape $\mathbb{R}^8$. A binary classifier determines whether each triplet maintains temporal order, and the Binary Cross-Entropy (BCE) loss is used to optimize the model:

$$L_{\text{Temp}} = (1 - o_n) \log(1 - \hat{o}_n) + o_n \log(\hat{o}_n) \quad (11)$$

where n refers to the index of the sample, $o_n \in \{0, 1\}$ is a ground-truth label indicating whether the triplet is in the correct temporal order (1 for ordered, 0 for disordered), while $\hat{o}_n \in \{0, 1\}$ is the predicted probability (output of a sigmoid layer) that the triplet is correctly ordered. This approach ensures that the embeddings reflect the temporal order of the sequences, enhancing the temporal accuracy and relevance of the features extracted by the Mamba encoder.

3.4.2. N-tuplet loss

The N-tuplet loss, another loss function used to capture sequential dependencies, enhances the model's ability to discriminate between multiple data segments by comparing a positive example against multiple negative examples within a batch [36]. As illustrated in Fig. 4(b), this approach aims to improve the discriminative ability of the embeddings and enhance model performance in distinguishing between different data conditions.

The N-tuplet loss function involves a query sample x , a positive sample (immediate next time step) x^+ , and $N-2$ negative samples (other in batch) $\{x_i\}_{i=1}^{N-2}$. The network learns an embedding function f that maps samples into a high-dimensional space. The objective is to minimize the loss function:

$$L_{N\text{-tuple}} = (\{x, x^+, \{x_i\}_{i=1}^{N-2}\}; f) = \log \left(1 + \sum_{i=1}^{N-2} \exp(f^\top f_i - f^\top f^+) \right) \quad (12)$$

This aims to maximize the inner product $f^\top f^+$ between the query and positive sample while minimizing the inner products $f^\top f_i$ between the query and each negative sample x_i . Here, $f^\top f^+$ denotes the dot product between the embeddings of the query and its temporally adjacent positive, encouraging temporal closeness, while $f^\top f_i$ measures similarity with unrelated segments, which the model is penalized for. This formulation effectively pulls temporally coherent representations together and pushes apart unrelated ones in the embedding space.

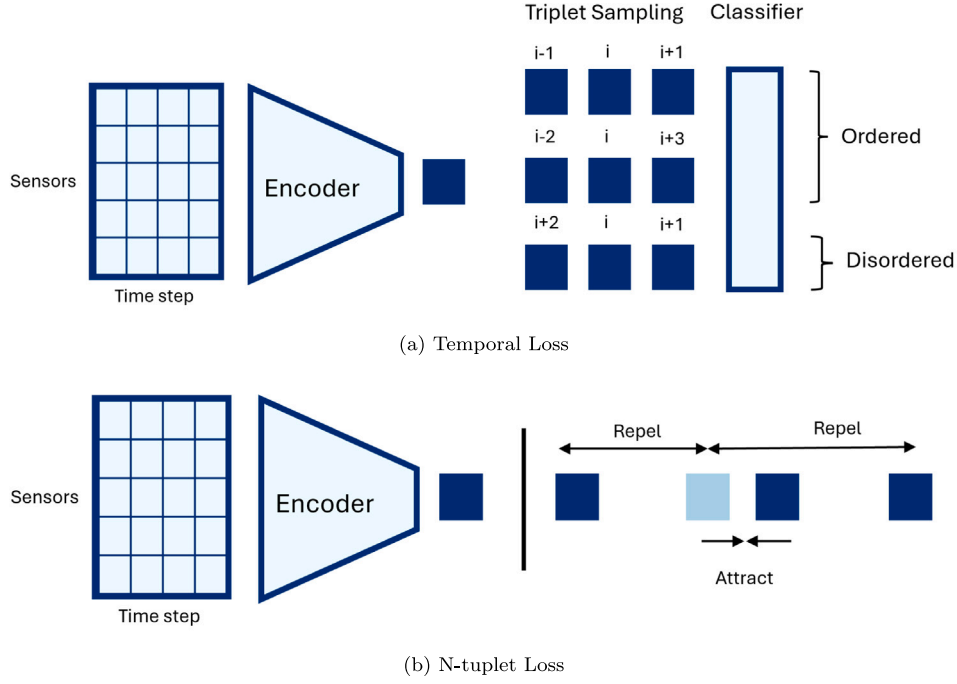


Fig. 4. Temporal and N-tuplet loss.

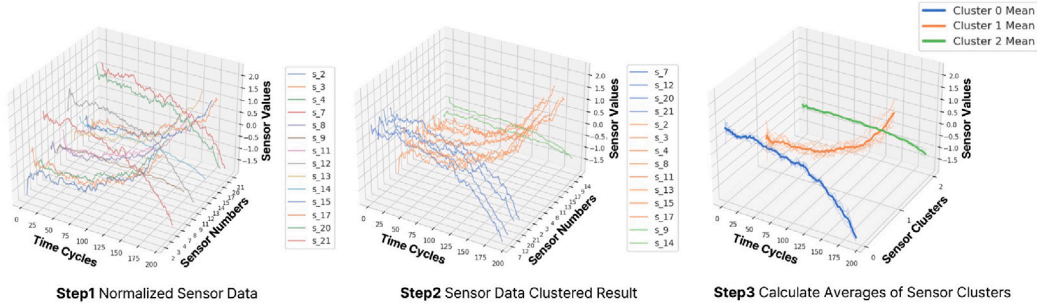


Fig. 5. Steps 1–3 of the pseudo-labeling process. Step 2 applies k-means clustering to segment sensor trajectories. See Algorithm 2 for full description.

3.4.3. Pseudo-label loss

To enable training in the absence of explicit labels, we adopt a structured pseudo-labeling approach that leverages characteristic patterns in the sensor signals. The process consists of seven systematic steps, as formally described in Algorithm 2.

In Steps 1–3 (illustrated in Fig. 5), non-informative sensors — those exhibiting minimal variation or lacking degradation trends — are excluded. The remaining signals are normalized and then clustered using k-means [37] ($k = 3$) to segment each sensor's trajectory into three distinct behavioral groups. The average value (centroid) of each cluster is then computed to facilitate boundary selection. We adopted k-means clustering for auxiliary pseudo-label generation owing to its simplicity, computational efficiency, and practical effectiveness in distinguishing sensor signal patterns.

Steps 4–6 (illustrated in Fig. 6) involve identifying two decision thresholds based on the computed cluster centroids. The first threshold (b_1) is derived from the boundary between clusters 2 and 3, and the second threshold (b_2) is computed as the midpoint between the sensor's minimum value and b_1 . These thresholds are used to assign stage-wise pseudo-labels (Label 1, 2, or 3) to each (window, sensor) instance.

Finally, in Step 7, a neural classifier is trained using cross-entropy loss to predict these pseudo-labels, guiding the model to learn a structured representation of sensor degradation patterns. The overall procedure is summarized in Algorithm 2.

Table 1

Description of the C-MAPSS Dataset.

	FD001	FD002	FD003	FD004
Training engines	100	260	100	249
Training samples	15 041	39 512	18 492	45 513
Testing engines	100	259	100	248
Working conditions	1	6	1	6
Fault types	1	1	2	2

While pseudo-labeling is effective in datasets with clear degradation (e.g., C-MAPSS), our framework remains robust using only temporal and contrastive losses when such trends are weak or absent, as shown in CALCE results in Section 5.3.2.

4. Experiments

4.1. C-MAPSS dataset

In this study, we employ the RUL C-MAPSS dataset [38], a standard benchmark provided by NASA, to evaluate the performance of prognostic models in predicting aerospace engine degradation. As shown in Table 1, this dataset simulates various operational and fault conditions

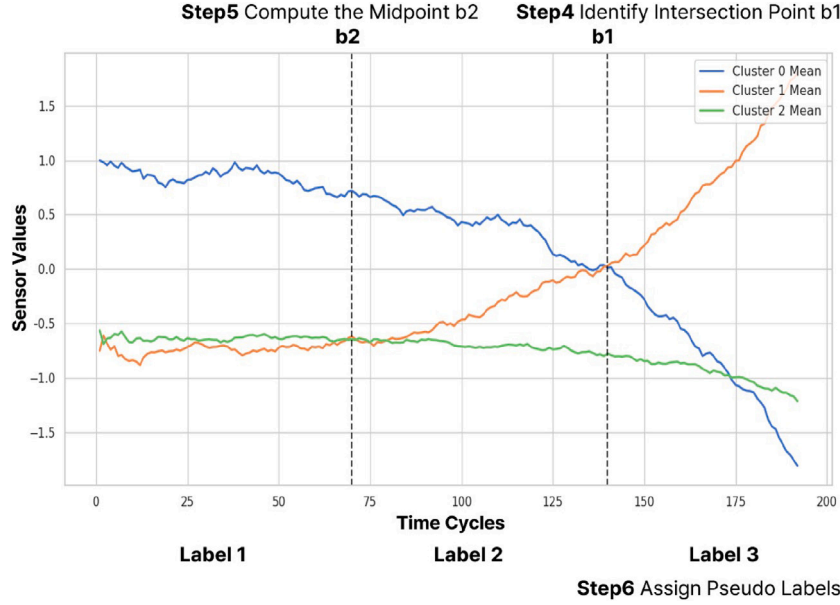


Fig. 6. Steps 4–6 of the pseudo-labeling process. Step 4 defines the threshold b_1 between cluster centroids. Labels are then assigned based on thresholds in Step 6.

Algorithm 2 Pseudo-Label Generation Procedure

- 1: **Input:** Unlabeled sensor time series data from C-MAPSS
 - 2: **Output:** Pseudo-labels for each (window, sensor) instance
 - 3: **Step 1:** Normalize all sensor values and exclude those that show no significant variation or abnormal behavior (sensors 1, 5, 6, 10, 16, 18, and 19)
 - 4: **for** each remaining sensor **do**
 - 5: **Step 2:** Apply k-means clustering ($k = 3$) to segment the data into three clusters
 - 6: **Step 3:** Compute the average value (centroid) of each cluster
 - 7: **Step 4:** Identify the intersection point between clusters 2 and 3 using their centroids, and define it as the first boundary (b_1)
 - 8: **Step 5:** Compute the midpoint between the sensor's minimum value and b_1 , and define it as the second boundary (b_2)
 - 9: **Step 6:** Assign pseudo-labels based on b_2 and b_1 :
 - Label 1: $x < b_2$
 - Label 2: $b_2 \leq x < b_1$
 - Label 3: $x \geq b_1$
 - 10: **end for**
 - 11: **Step 7:** Train a neural network classifier to predict the pseudo-labels using cross-entropy loss
-

of turbofan engines under different scenarios, enabling the development and testing of predictive maintenance algorithms. Specifically, we utilize the multiple sub-datasets (FD001 to FD004), which differ in complexity based on varying operating conditions and fault modes. Each instance in the dataset includes 21 sensor readings, representing parameters such as temperature, pressure, and rotational speed, which are critical engine parameters, offering the necessary data to model and predict the engine's RUL. This dataset validates our model's ability to learn temporal patterns and accurately predict the remaining operational cycles before engine failure.

4.1.1. RUL calculation for the C-MAPSS dataset

$$RUL(t) = \text{Cycle}_{\max} - t \quad (13)$$

where, $RUL(t)$ denotes the remaining useful life value at time t , $\text{Cycle}_{\max}$ represents the maximum cycle number for each engine sample, and t is the elapsed time. The RUL decreases by one unit with each time unit that passes, starting from the maximum cycle value.

4.1.2. C-MAPSS dataset pre-processing

Piece-wise linear degradation

In the C-MAPSS dataset, sensor values remain constant until a certain degradation threshold is reached. Following the approach adopted by [11], we set all RUL values greater than 125 to 125. This adjustment reflects the piece-wise linear degradation patterns observed in the dataset shown in Fig. 7.

Sliding window

To capture temporal dependencies in the time-series data of the C-MAPSS dataset, we apply a sliding window approach. This allows the model to learn from multiple time points rather than a single time point. The sliding stride is set to 1, and each window contains data samples from the same engine unit to maintain consistency.

Sensor selection

We utilized all 21 sensors from the C-MAPSS dataset, as MambAtt demonstrated strong performance without requiring a separate sensor selection process. However, for efficient clustering in the pseudo-label Loss, we removed sensors that were either constant or irrelevant. See 3.4.3 for details.

Normalization

To ensure balanced learning and improve training stability and model performance, we applied Z-score normalization to the sensor data to prevent certain features from dominating due to scale differences. Given that FD001 and FD003 have one working condition, while

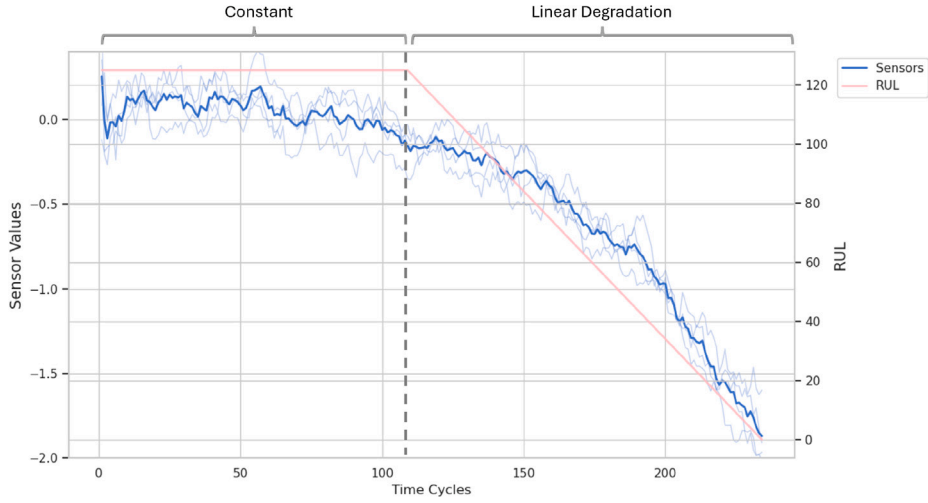


Fig. 7. Piece-wise linear degradation remaining useful life in C-MAPSS dataset.

FD002 and FD004 have six, we additionally performed normalization per sensor and condition for FD002 and FD004.

The Z-score normalization for each sensor s and working condition w is applied using the following formula:

$$z_{s,w} = \frac{x_{s,w} - \mu_{s,w}}{\sigma_{s,w}} \quad (14)$$

where $x_{s,w}$ represents the raw sensor value, $\mu_{s,w}$ is the mean, and $\sigma_{s,w}$ is the standard deviation of sensor s under working condition w . This normalization ensures that each sensor's data is centered around zero with a standard deviation of one, allowing for balanced feature scaling across different conditions.

Finally, using the window size w and the number of sensors s (resulting in an input of $w \times s$), we perform RUL prediction using the target values calculated through piece-wise linear degradation.

4.2. CALCE dataset

To assess the generalization capability of our model, we additionally performed experiments using the publicly well-known CALCE Lithium-ion (Li-ion) battery dataset provided by the University of Maryland [39]. This dataset, which differs from the C-MAPSS aircraft engine dataset in domain characteristics, includes four battery models: CS2_35, CS2_36, CS2_37, and CS2_38. These batteries were tested at a constant temperature of 1 °C. We treated each of the four battery datasets as a distinct engine type and performed RUL prediction accordingly. The CALCE dataset, typical of long-life batteries in battery life research [24], differs from the C-MAPSS dataset in that it generally contains batteries with over 800 cycles on average. Each battery undergoes a sequence of three operations: charging, discharging, and impedance measurements.

4.2.1. RUL calculation for the CALCE dataset

In Li-ion battery RUL prediction, capacity value is broadly regarded as the health indicator of a battery to quantify degradation. A battery reaches its End Of Life (EOL) threshold when its capacity is reduced to seventy or eighty percent of the initial value [40]. RUL is defined as the amount of time remaining before system health falls below a pre-determined failure threshold.

Similar to previous studies [40–42] that performed RUL prediction using the CALCE battery dataset, our approach forecasts future capacity values to enable indirect inference of remaining useful life based on degradation trends.

4.2.2. CALCE dataset pre-processing

Following the training data setup used in [40,41], we applied **Normalization** and **Sliding window** (with a window size of 64 and a stride of 1) to the input capacity sequences, as described earlier. Raw input data often contains much noise, particularly during charge/discharge regeneration, which has a detrimental effect on the accuracy of predictions [40]. We additionally applied a dropout-based outlier removal process.

Dropout outlier

The input sequence is divided into non-overlapping segments of fixed size (bins), and within each segment, data points that fall outside two standard deviations from the mean are considered outliers and excluded. The remaining indices are retained and used as valid input for training.

4.3. Evaluation metrics

To evaluate the performance of our model, we used various metrics: Root Mean Square Error (RMSE), Mean Absolute Error (MAE), SCORE, and Relative Error (RE) are defined as follows.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (15)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n \|y_i - \hat{y}_i\| \quad (16)$$

$$\text{SCORE} = \sum_{i=1}^n \begin{cases} \exp\left(\frac{-\hat{e}_i}{13}\right) - 1 & \text{if } \hat{e}_i < 0 \\ \exp\left(\frac{\hat{e}_i}{10}\right) - 1 & \text{if } \hat{e}_i \geq 0 \end{cases} \quad (17)$$

The SCORE metric is specifically designed to evaluate the performance of RUL prediction models [38]. This metric considers both underestimation and overestimation of the RUL. This metric assigns a higher penalty to underestimate the RUL, as it can lead to premature maintenance actions and increased costs, while overestimation may result in unexpected failures. The asymmetric nature of the RUL score makes it particularly suitable for safety-critical applications, where the consequences of failure due to underestimation are much more severe than those of unnecessary maintenance caused by overestimation.

5. Experiments and results

In this study, we focus on two primary tasks: First, we utilize the MambaAtt model to perform traditional RUL prediction, leveraging its

Table 2
Training hyperparameters of C-MAPSS and CALCE datasets.

Hyperparameter	Datasets	
	C-MAPSS	CALCE
Learning rate	$1e^{-3}$	$1e^{-3}$
Window size	20	64
Batch size	64	64
Mamba_d_state	16	16
Mamba_d_conv	8	4
Number of transformer layers	3	2
Number of attention heads	7	4
Optimizer	Adam	Adam
Dropout rate	0.5	0.1
Number of iterations	50	5

architecture to enhance the accuracy and robustness of the prognostic outcomes. Second, we implement a self-supervised learning approach by pre-training the encoder component of MambaAtt using unlabeled data and three self-supervised learning losses. The pre-trained weights are subsequently fine-tuned through transfer learning, enabling the model to effectively perform self-supervised tasks with minimal labeled data in a one-shot learning scenario. These two approaches not only demonstrate the versatility of MambaAtt in RUL prediction but also verify its potential in learning from limited labeled data through advanced self-supervised techniques.

5.1. Experimental settings

The experimental setup followed the configuration presented in Table 2. Following the experimental settings outlined in [22], we implement multiple random seeds to ensure robustness and prevent the model from converging to a local minimum. The model is trained 50 or 5 times on each of the four sub-datasets using different random seeds. Subsequently, the model exhibiting the lowest RMSE on the validation set is selected and evaluated on the test set. In the few-shot learning scenario, for C-MAPSS dataset, a small number of samples are randomly selected to reflect real-world conditions. One of these is further designated as a validation sample to determine the optimal training epoch. For the CALCE dataset, the battery CS2_35 is used as the one-shot test instance. In the two-shot setting, we conducted additional experiments using various combinations of labeled samples to assess the consistency and generalizability of the model. Since the CALCE dataset does not exhibit a clear degradation trend suitable for pseudo-labeling, we pre-trained the model using only the temporal and N-tuplet losses.

For validation purposes in C-MAPSS dataset experiments, 80% of the engines are designated as the training set, while the remaining 20% are used as the validation set. For the CALCE battery dataset, we adopted a leave-one-out evaluation strategy. In each iteration, one battery was designated as the test sample, while the others were used for training. This procedure was repeated five times, and the final results were reported as the average performance across all batteries.

5.2. RUL prediction in supervised learning (full-shot)

5.2.1. Result on C-MAPSS dataset

As shown in Table 3, state-of-the-art benchmark models are selected as references to better illustrate the overall performance of the proposed MambaAtt model under a fully supervised learning setting using complete RUL labels. This setting is consistent with those used in prior RUL prediction studies, ensuring fair and comparable evaluation. The table includes various models: RNN-based models (D-LSTM [10], BS-LSTM [43], RF-LSTM [44]), attention-based models (DA-Transformer [7], GCU-Transformer [14], IDMFNN [13], GA-Transformer [48]), Attention-RNN based models (AM-BGRU [46], BIGRU-TSAM [19], AM-LSTM [45], NTM [47], DA-LSTM [22]) and a model integrating contrastive learning approaches (CBHRL [27]). To

reflect recent trends in state space modeling, we also conducted experiments with S4 [32], an SSM model. However, its performance degrades in complex scenarios like FD004 due to limited adaptability to non-stationary patterns. The performance of these models across the FD001 to FD004 sub-datasets is compared using RMSE and SCORE metrics. In the comparison tables, the best-performing models are highlighted in bold red, indicating first place, while the second-best results are marked with an underline blue, indicating second place. The MambaAtt model demonstrates superior performance across multiple sub-datasets, particularly excelling in FD002 and FD004, where it achieves the lowest RMSE values. Specifically, MambaAtt attains an RMSE of 13.20 on FD002 and 13.25 on FD004, outperforming other models. In FD001 and FD003, the model achieved second place, demonstrating RMSE values very close to those of the first-place models. Similarly, the SCORE metric further emphasizes the MambaAtt model's effectiveness. The model achieves the SCORE of 717 on FD002, 179 on FD003, and 1114 on FD004 outperforming the comparison models.

5.2.2. Result on CALCE dataset

As shown in Table 4, our method outperforms all baseline models on the CALCE dataset in terms of RMSE and MAE, achieving the lowest RMSE of 0.0683 and the lowest MAE of 0.0551. These findings indicate that our model achieves robust and superior performance across different domains, as evidenced by its effectiveness on both the C-MAPSS and CALCE datasets.

5.3. RUL prediction in self-supervised learning (few-shot)

5.3.1. Result on C-MAPSS dataset

Qualitative analysis of self-supervised embeddings

Fig. 8 illustrates the embeddings of the engine samples derived from FD001 sub-dataset after pre-training with self-supervised learning using MambaAtt. In Fig. 8(a), these embeddings are visualized in two dimensions using t-SNE [49] for dimensionality reduction. The resulting embeddings display a sequential pattern from large to small RUL values, indicating the model's ability to capture the degradation process. Similarly, Fig. 8(b) presents a 3D visualization of the embeddings after dimensionality reduction with t-SNE. The blue points in this figure represent the second engine sample from the FD001, while the yellow points correspond to the third engine sample. The embeddings for both samples are located in similar positions, with larger RUL values clustered together and smaller RUL values likewise positioned in proximity, exhibiting a sequential pattern.

One-shot learning

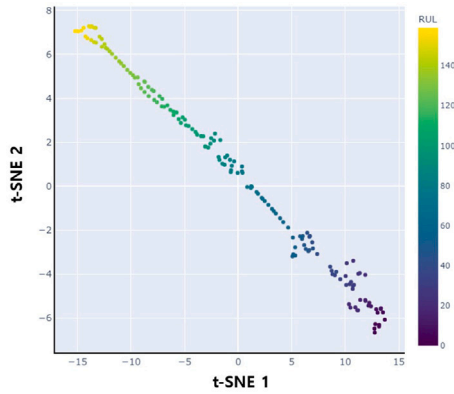
Table 5 compares the results of embeddings generated using the GRU, Transformer, and MambaAtt models in a one-shot learning task. In this task, the models are pre-trained using a self-supervised learning method and then trained with only a single label. Except for the FD004 dataset, where the Transformer exhibited better performance, MambaAtt consistently outperformed the other models. This confirms that the MambaAtt model is effective when the model is trained using the self-supervised learning approach. Interestingly, in FD004, which involves more complex operating conditions and multiple fault types, the Transformer slightly outperforms MambaAtt. This may suggest the advantage of global attention in modeling highly heterogeneous data. Nevertheless, MambaAtt maintains competitive performance across all subsets.

Few-shot learning

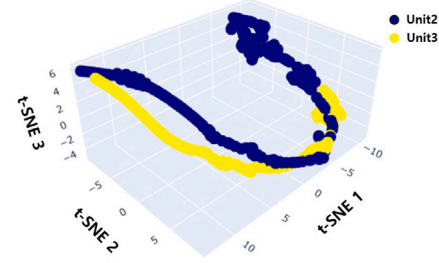
Figs. 9 and 10 present error line charts that illustrate the performance of few-shot learning when fine-tuning is conducted with a limited number of labeled samples. The central lines in these figures represent the average performance for each number of shots, while the shaded regions surrounding the central lines indicate the 95% confidence intervals, reflecting variability. Fig. 9 visualizes the RMSE

Table 3
Comparison of RUL prediction performance under fully supervised training on C-MAPSS dataset.

Methods	RMSE				SCORE			
	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
D-LSTM [10], 2017	16.14	24.49	16.18	28.17	338	4450	852	5550
BS-LSTM [43], 2018	14.89	26.86	15.11	27.11	481	7982	493	5200
RF-LSTM [44], 2020	16.87	23.91	17.89	25.49	N/A	N/A	N/A	N/A
AM-LSTM [45], 2020	14.53	N/A	N/A	27.08	322	N/A	N/A	5649
AM-BGRU [46], 2021	12.42	19.43	13.39	21.50	226	1492	227	3392
GCU-Transformer [14], 2021	11.27	22.81	11.42	24.86	N/A	N/A	N/A	N/A
NTM [47], 2022	12.53	27.04	13.73	28.11	210	6000	270	4800
BIGRU-TSAM [19], 2022	12.56	18.94	12.45	20.47	213	2264	233	3610
DA-Transformer [7], 2022	12.25	17.08	13.39	19.86	198	1575	290	1741
IDMFFN [13], 2023	12.18	19.17	11.89	21.72	205	1819	206	3339
GA-Transformer [48], 2023	11.63	15.99	11.35	20.15	215	1133	228	2672
DA-LSTM [22], 2024	12.62	13.22	13.34	16.25	263	842	360	1372
CBHRL [27], 2024	12.44	15.55	12.88	18.28	249	1215	258	1838
S4 [32], 2021	14.81	14.44	13.75	19.50	271	933	261	28513
MambAtt	11.46	13.20	11.56	13.25	201	717	179	1114



(a) 2D t-SNE visualization colored by RUL



(b) 3D t-SNE visualization for multiple units

Fig. 8. 2D and 3D t-distributed stochastic neighbor embedding (t-SNE) visualizations of the pre-trained embeddings.

Table 4

Comparison of RUL prediction performance under fully supervised training on CALCE dataset.

Methods	MLP	LSTM	GRU	Dual-LSTM	AttMoE [40]	Ours
RMSE	0.2038	0.0736	0.0946	0.0874	0.0794	0.0683
MAE	0.1557	0.0582	0.0671	0.0636	0.0577	0.0551

values for few-shot learning. The blue line represents the results when few-shot learning is applied directly without any self-supervised pre-training. In contrast, the orange line shows the results when pre-trained weights with self-supervised losses are fine-tuned for few-shot learning. In both scenarios, RMSE values decrease as the number of shots increases. Moreover, the confidence intervals narrow from one-shot to five-shot learning, indicating a reduction in variability. Evidently, RMSE values are significantly lower when self-supervised learning is performed compared to when it is not, demonstrating the effectiveness of self-supervised losses in few-shot learning. Fig. 10 presents the error line chart for the SCORE metric. Similar to the RMSE results, both self-supervised and non-self-supervised models exhibit a decrease in SCORE values as the number of shots increases. The application of self-supervised learning results in a significant reduction in SCORE values, along with a notable narrowing of the confidence intervals. This suggests that the use of self-supervised learning not only reduces SCORE but also decreases variance, leading to more robust outcomes.

5.3.2. Result on CALCE dataset

Table 6 presents the results of one-shot and two-shot RUL prediction on the CALCE battery dataset, comparing scenarios with and without

self-supervised pre-training (self-sup). Due to the characteristics of the CALCE dataset, which lacks clearly observable degradation stages, only the temporal and N-tuplet losses were used for self-supervised pre-training. In the table, a dash (–) in the “Self-sup” column indicates that no pre-training was applied. The row immediately following each dash presents the result of applying self-supervised pre-training using the unlabeled data configuration stated in that row. Metrics include root mean squared error (RMSE) and mean absolute error (MAE). Overall, self-supervised pre-training consistently demonstrates significant performance improvements over the baseline models trained without pre-training, indicating the efficacy of the proposed self-supervised framework. Particularly, the one-shot and two-shot settings highlight that even minimal labeled data, when combined with self-supervised learning, can substantially enhance predictive accuracy, underscoring the value of this approach for practical scenarios with limited labeled samples.

5.4. Ablation study

5.4.1. Comparison of RUL predictions

Fig. 11 visualizes the predicted RUL values for one engine from each of the FD001–FD004 datasets, comparing three variants: full-shot supervised learning, one-shot with self-supervised pretraining, and one-shot without self-supervised loss. Overall, the full-shot supervised model produces the most accurate predictions, closely following the actual RUL trajectory. The one-shot model with self-supervised loss also provides reasonably accurate results. In contrast, the one-shot model without self-supervision exhibits erratic and less reliable predictions.

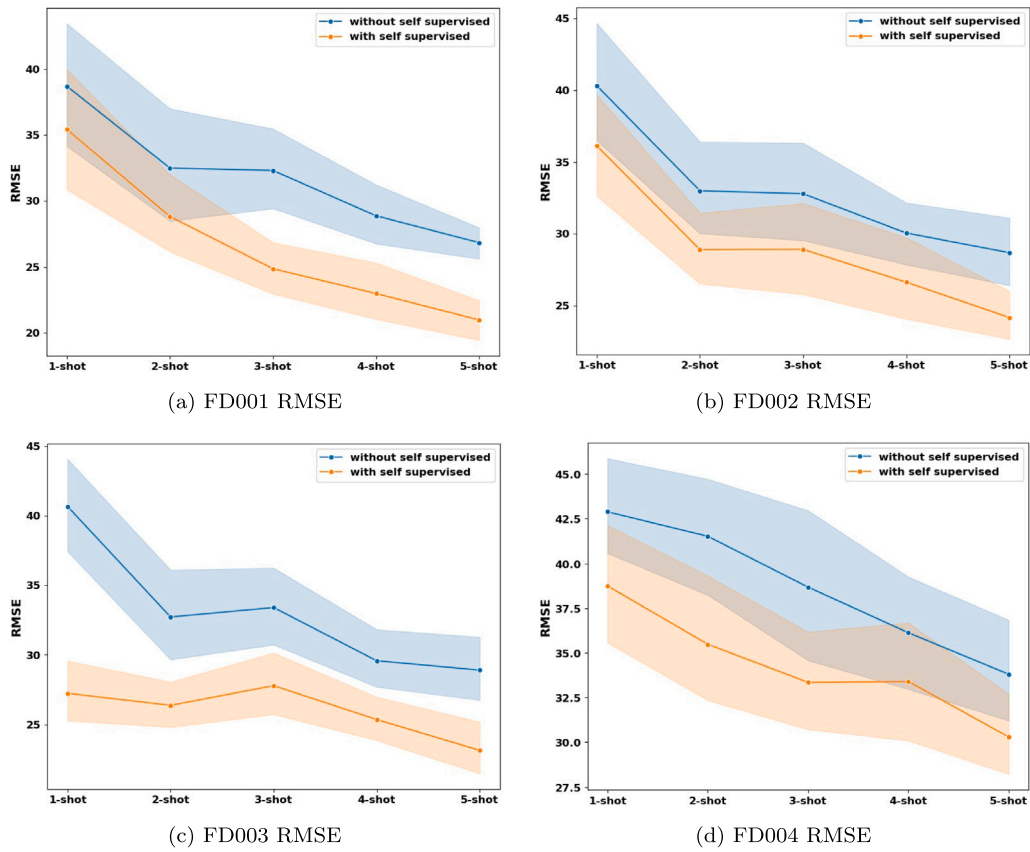


Fig. 9. RMSE comparison on the C-MAPSS dataset with and without self-supervised learning.

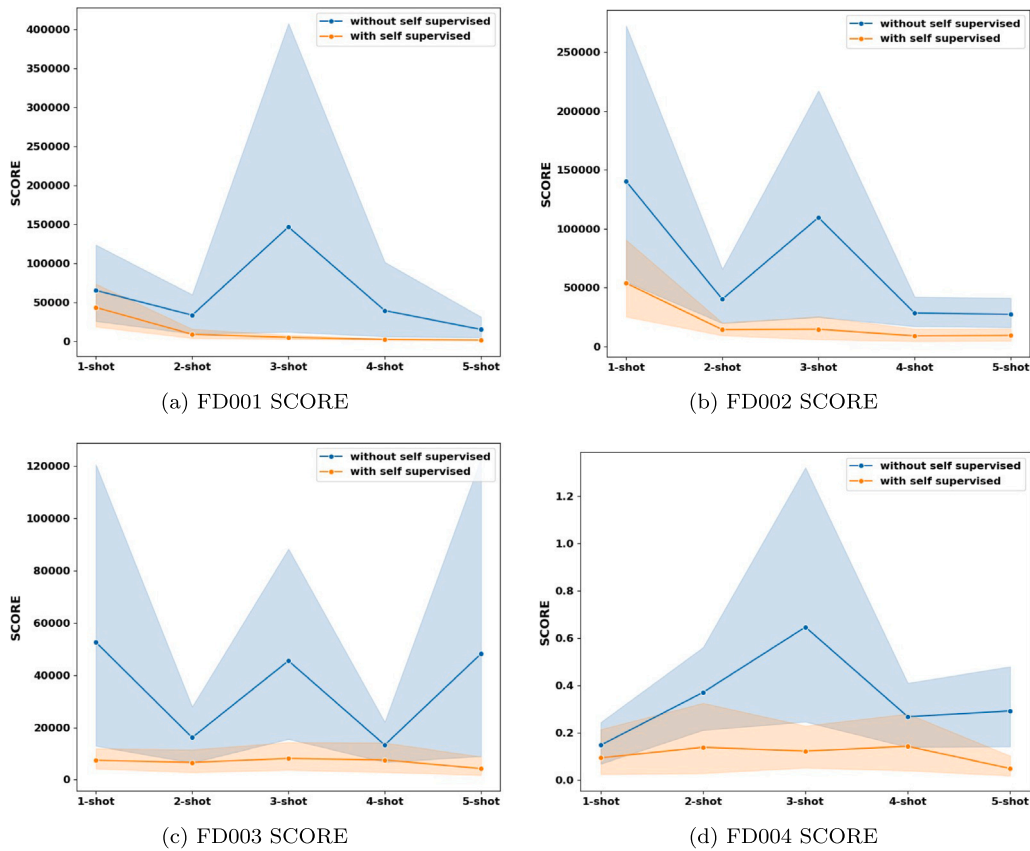


Fig. 10. Score comparison on the C-MAPSS dataset with and without self-supervised learning.

Table 5
Self-supervised learning results with one-shot learning.

Model	RMSE				SCORE			
	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
GRU + Self-sup	53.721	83.4962	83.4068	83.2265	1 080 056	6 025 718	5 968 604	6 020 216
Transformer + Self-sup	36.189	33.8233	42.2206	27.8217	18 872	15 228	47 792	4456
Mamba + Self-sup	31.727	30.3269	32.3329	32.6633	5086	2831	6438	7079

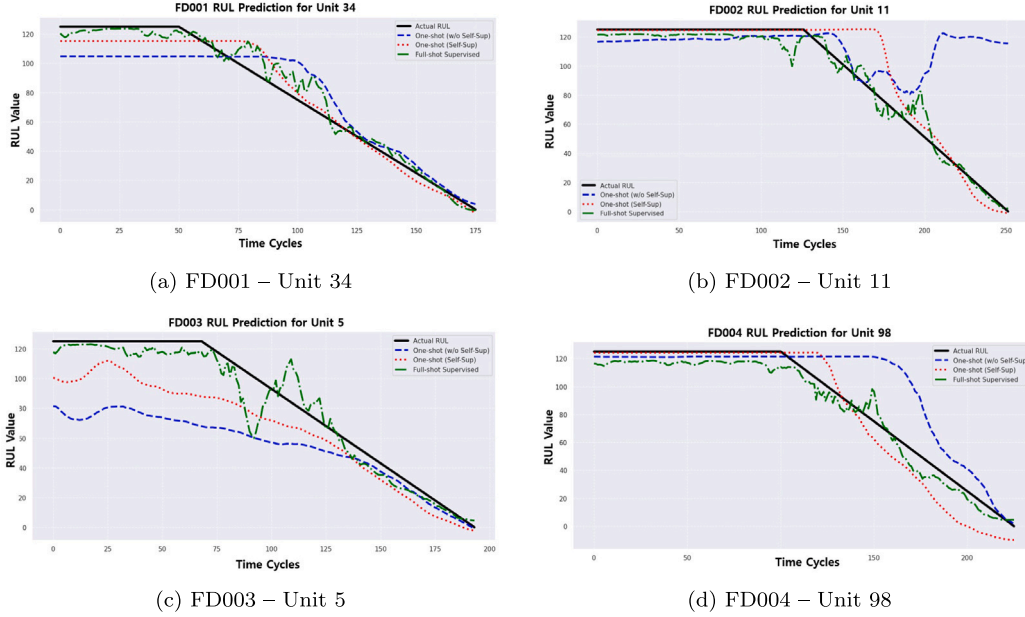


Fig. 11. Comparison of RUL predictions for representative units.

Table 6

Few-shot RUL prediction performance on CALCE dataset with and without self-supervised pre-training (self-sup).

Setting	Self-supervised learning data configuration			Performance	
	Self-sup (Unlabeled)	Supervised (Labeled)	Target (Test)	MAE	RMSE
One-shot	–	CS2_36	CS2_35	0.1378	0.1907
	CS2_37, CS2_38	CS2_36	CS2_35	0.0967	0.1216
	–	CS2_37	CS2_35	0.1180	0.1647
	CS2_36, CS2_38	CS2_37	CS2_35	0.0738	0.0952
	–	CS2_38	CS2_35	0.0758	0.1036
	CS2_36, CS2_37	CS2_38	CS2_35	0.0698	0.0886
Two-shot	–	CS2_37, CS2_38	CS2_35	0.1304	0.1743
	CS2_36	CS2_37, CS2_38	CS2_35	0.0659	0.0925
	–	CS2_35, CS2_36	CS2_37	0.1304	0.1743
	CS2_38	CS2_35, CS2_36	CS2_37	0.0661	0.0916
	–	CS2_36, CS2_37	CS2_38	0.1209	0.1687
	CS2_35	CS2_36, CS2_37	CS2_38	0.0914	0.1183

This illustrates the benefit of structured representation learning even in low-label regimes.

Notably, predictions from the one-shot model — particularly when self-supervised pretraining is applied — exhibit smoother trajectories, whereas the full-shot model occasionally overfits to local fluctuations. This behavior may stem from the fact that full-shot training utilizes all available labeled instances, enabling the model to capture fine-grained patterns including outliers, while the one-shot setting tends to prioritize more stable patterns rather than overfitting to specific training instances.

5.4.2. Training time

Fig. 12 presents bar charts comparing the training durations of GRU, Transformer, and MambAtt models using C-MAPSS dataset. Fig. 12(a) presents the pre-training times using self-supervised learning losses calculated in seconds. MambAtt required the shortest training time in all datasets. This demonstrates that MambAtt's parallel computation and hardware-aware algorithm are efficient and effectively reduce training time. In Fig. 12(b), the times taken by each model when performing one-shot learning 10 times are calculated in seconds. The MambAtt model shows a significant reduction in time, ranging from 64.5% to 82.8%, compared to the GRU and Transformer models.

5.4.3. Variation of decoder model

In the proposed MambAtt architecture, we utilized a Transformer block layer as the final decoder to extract the RUL value. This decision arose from investigating several decoder options specifically, a Fully Connected Layer, a CLS token approach (which appends a special “CLS” token vector to the Mamba output embeddings before applying a Fully Connected Layer), a GRU layer, and a Transformer block layer. As shown in Table 7, we conducted 5 independent experiments for C-MAPSS dataset and averaged the RMSE and Score metrics on the test set. The results indicate that the Transformer block layer outperforms the other decoders, highlighting its effectiveness at capturing the temporal dependencies crucial for accurate RUL estimation.

5.4.4. Self-supervised loss combination test

Table 8 presents the results of a one-shot learning ablation study on self-supervised losses, conducted with a fixed sample of the 12th engine for training and the 84th engine for validation. For the FD001, FD002, and FD003, the RMSE performance improved when both the temporal

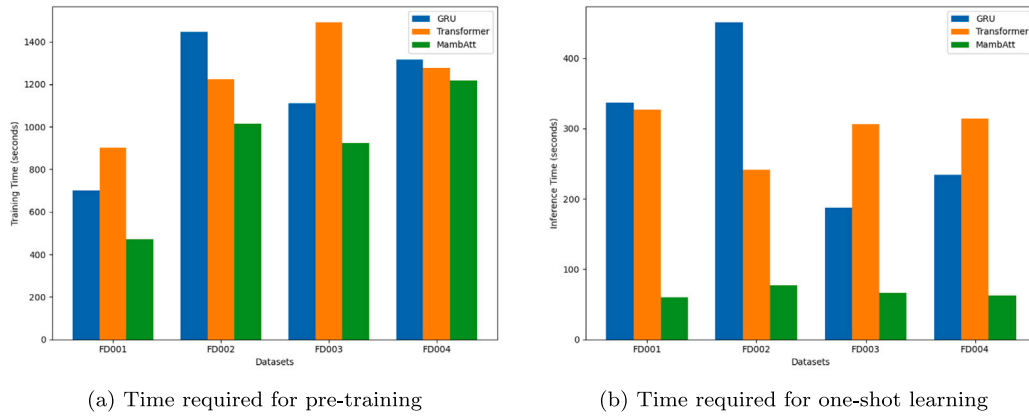


Fig. 12. Comparison of training time.

Table 7

Decoding layer results for RUL prediction.

Mamba + Decoder	RMSE				SCORE			
	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
Fully connected layer	14.0353	15.8201	13.3451	17.9534	311.8749	2981.0234	244.5646	270175.5812
CLS token + Fully connected layer	14.1862	16.2340	14.3046	18.2760	356.3304	1451.2500	359.0417	36812.3141
GRU	12.8672	13.6014	12.8271	15.9190	248.6681	757.1132	264.9265	15716.2647
Transformer	12.3409	13.6121	12.8798	14.6383	252.7097	780.4995	194.7736	1551.3023

Table 8

Ablation study on self-supervised learning losses.

Temp	N-tuplet	Pseudo	RMSE				SCORE			
			FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
✓			32.60	33.88	29.98	31.34	5933	22 942	16 370	20 971
✓	✓		31.02	32.84	29.56	43.40	4556	26 258	7442	21 430
✓	✓	✓	28.35	27.57	25.67	43.42	3246	10 322	7749	21 500

and N-tuplet losses were used in a 0.5:0.5 ratio compared to when only the temporal loss was used. The best performance was achieved when all three losses were employed in the ratio of 0.25:0.25:0.5. Similarly, for the SCORE metric, using all three losses yielded the best results in FD001 and FD002. However, in FD003, the performance difference between using both temporal and N-tuplet losses and using all three losses was minimal. Conversely, in the case of FD004, the best results were obtained when only the temporal loss was used. This suggests that the temporal loss is particularly effective for complex and large datasets.

6. Conclusion

In this study, we proposed Mamba-Attention (MambAtt) model, which overcomes the limitations of traditional RNNs and attention-based models, such as vanishing gradients, inefficiencies in long sequence processing, and high computational complexity. By integrating a selective state space model with attention mechanisms, MambAtt achieved state-of-the-art performance in RUL prediction tasks while improving training speed. We also explored the use of self-supervised learning (SSL) and few-shot learning approaches. With SSL techniques: temporal triplet loss, N-tuplet loss, and pseudo-label loss, the model effectively generated accurate RUL predictions with minimal labeled data. Few-shot learning further demonstrated the framework's robustness, though challenges remain, particularly in performance variability based on the chosen few-shot labels. Future research should focus on reducing this variability to enhance the stability and reliability of few-shot learning in RUL prediction.

CRediT authorship contribution statement

Jihun Han: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization. **Yejin Kwon:** Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation. **Hyunsoo Yoon:** Writing – review & editing, Supervision, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2025-02305884) and by the National Research Foundation of Korea (NRF), South Korea grant funded by the Korean government (RS-2023-00213798).

Data availability

Data utilized in this study are publicly accessible. Please refer to <https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data> and <https://calce.umd.edu/battery-data>.

References

- [1] Compare M, Bellani L, Zio E. Reliability model of a component equipped with PHM capabilities. *Reliab Eng Syst Saf* 2017;168:4–11.
- [2] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. *Mech Syst Signal Process* 2018;104:799–834.
- [3] Han X, Wang Z, Xie M, He Y, Li Y, Wang W. Remaining useful life prediction and predictive maintenance strategies for multi-state manufacturing systems considering functional dependence. *Reliab Eng Syst Saf* 2021;210:107560.
- [4] Zhang Q, Liu Q, Ye Q. An attention-based temporal convolutional network method for predicting remaining useful life of aero-engine. *Eng Appl Artif Intell* 2024;127:107241.
- [5] Zhang K, Wen Q, Zhang C, Cai R, Jin M, Liu Y, Pan S. Self-supervised learning for time series analysis: Taxonomy, progress, and prospects. *IEEE Trans Pattern Anal Mach Intell* 2024.
- [6] Wang Y, Ming H, Zhang G, Ai X, Xu F, Li B. Research on fault diagnosis system based on aeroengine knowledge base. In: 2022 global reliability and prognostics and health management (PHM-yantai). 2022, p. 1–5.
- [7] Liu L, Song X, Zhou Z. Aircraft engine remaining useful life estimation via a double attention-based data-driven architecture. *Reliab Eng Syst Saf* 2022;221:108330.
- [8] Wahid A, Yahya M, Breslin JG, Intizar MA. Self-attention transformer-based architecture for remaining useful life estimation of complex machines. *Procedia Comput Sci* 2023;217:456–64.
- [9] Babu SG, Zhao P, Li XL. Deep convolutional neural network based regression approach for estimation of remaining useful life. In: Database systems for advanced applications: 21st international conference, DASFAA 2016, Dallas, TX, USA, April (2016) 16–19, proceedings, part i 21. Springer International Publishing; 2016, p. 214–28.
- [10] Zheng S, Ristovski K, Farahat A, Gupta C. Long short-term memory network for remaining useful life estimation. In: 2017 IEEE international conference on prognostics and health management. ICPHM, 2017, p. 88–95.
- [11] Liu H, Liu Z, Jia W, Lin X. A novel deep learning-based encoder–decoder model for remaining useful life prediction. In: 2019 international joint conference on neural networks. IJCNN, 2019, p. 1–8.
- [12] Mo H, Custode LL, Iacca G. Evolutionary neural architecture search for remaining useful life prediction. *Appl Soft Comput* 2021;108:107474.
- [13] Zhang J, Li X, Tian J, Luo H, Yin S. An integrated multi-head dual sparse self-attention network for remaining useful life prediction. *Reliab Eng Syst Saf* 2023;233:109096.
- [14] Mo Y, Wu Q, Li X, Huang B. Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. *J Intell Manuf* 2021;32(7):1997–2006.
- [15] Gu A, Dao T. Mamba: Linear-time sequence modeling with selective state spaces. 2023, arXiv preprint arXiv:2312.00752.
- [16] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Comput* 1997;9(8):1735–80.
- [17] Cho K, Van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, Bengio Y. Learning phrase representations using RNN encoder–decoder for statistical machine translation. 2014, arXiv preprint arXiv:1406.1078.
- [18] Chen J, Jing H, Chang Y, Liu Q. Gated recurrent unit based recurrent neural network for remaining useful life prediction of nonlinear deterioration process. *Reliab Eng Syst Saf* 2019;185:372–82.
- [19] Zhang J, Jiang Y, Wu S, Li X, Luo H, Yin S. Prediction of remaining useful life based on bidirectional gated recurrent unit with temporal self-attention mechanism. *Reliab Eng Syst Saf* 2022;221:108297.
- [20] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Polosukhin I. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [21] Chang Y, Li F, Chen J, Liu Y, Li Z. Efficient temporal flow transformer accompanied with multi-head probsparse self-attention mechanism for remaining useful life prognostics. *Reliab Eng Syst Saf* 2022;226:108701.
- [22] Shi J, Zhong J, Zhang Y, Xiao B, Xiao L, Zheng Y. A dual attention LSTM lightweight model based on exponential smoothing for remaining useful life prediction. *Reliab Eng Syst Saf* 2024;243:109821.
- [23] Patro BN, Agneeswaran VS. Mamba-360: Survey of state space models as transformer alternative for long sequence modelling: Methods, applications, and challenges. 2024, arXiv preprint arXiv:2404.16112.
- [24] Wang HK, Gao M, Dai X, Cui L. A multi-frequency feature extraction and sparse attention mechanism integrated Mamba model for lithium-ion battery state of health estimation. *J Energy Storage* 2025;123:116643.
- [25] Shi Z. MambaLithium: Selective state space model for remaining-useful-life, state-of-health, and state-of-charge estimation of lithium-ion batteries. 2024, arXiv preprint arXiv:2403.05430.
- [26] Liu X, Zhang F, Hou Z, Mian L, Wang Z, Zhang J, Tang J. Self-supervised learning: Generative or contrastive. *IEEE Trans Knowl Data Eng* 2021;35(1):857–76.
- [27] Zhu Q, Zhou Z, Li Y, Yan R. Contrastive BiLSTM-enabled health representation learning for remaining useful life prediction. *Reliab Eng Syst Saf* 2024;249:110210.
- [28] Wang Y, Shen L, Zhang Y, Li Y, Zhang R, Yang Y. Self-supervised health representation decomposition based on contrast learning. *Reliab Eng Syst Saf* 2023;239:109455.
- [29] Yu Z, Lei N, Mo Y, Xu X, Li X, Huang B. Feature extraction based on self-supervised learning for remaining useful life prediction. *J Comput Inf Sci Eng* 2024;24(2).
- [30] Gu A, Johnson I, Goel K, Saab K, Dao T, Rudra A, Ré C. Combining recurrent, and continuous-time models with linear state space layers. *Adv Neural Inf Process Syst* 2021;34:572–85.
- [31] Pechlivanidou G, Karapetakis N. Zero-order hold discretization of general state space systems with input delay. *IMA J Math Control Inform* 2022;39(2):708–30.
- [32] Gu A, Goel K, Ré C. Efficiently modeling long sequences with structured state spaces. 2021, arXiv preprint arXiv:2111.00396.
- [33] Gu A, Dao T, Ermon S, Rudra A, Ré C. Hippo: Recurrent memory with optimal polynomial projections. *Adv Neural Inf Process Syst* 2020;33:1474–87.
- [34] Elfving S, Uchibe E, Doya K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Netw* 2018;107:3–11.
- [35] Misra I, Zitnick CL, Hebert M. Shuffle and learn: unsupervised learning using temporal order verification. In: Computer vision–ECCV 2016: 14th European conference, Amsterdam, the Netherlands, October (2016) 11–14, proceedings, part i 14. Springer International Publishing; 2016, p. 527–44.
- [36] Sohn K. Improved deep metric learning with multi-class n-pair loss objective. *Adv Neural Inf Process Syst* 2016;29.
- [37] Hartigan JA, Wong MA. Algorithm AS 136: A k-means clustering algorithm. *J R Stat Soc Ser C Appl Stat* 1979;28(1):100–8.
- [38] Coble JB, Hines JW. Prognostic algorithm categorization with PHM challenge application. In: 2008 international conference on prognostics and health management. IEEE; 2008, p. 1–11.
- [39] Vasan ASS, Mahadeo DM, Doraiswami R, Huang Y, Pecht M. Center for Advanced Life Cycle Engineering (CALCE), University of Maryland, College Park, Maryland 20742, USA, 2Center for Prognostics and system health management, city University of Hong Kong, 83 Tat Chee Avenue, Kowloon, Hong Kong. *Front Biosci* 2013;5:39–71.
- [40] Chen D, Zhou X. AttMoE: Attention with Mixture of Experts for remaining useful life prediction of lithium-ion batteries. *J Energy Storage* 2024;84:110780.
- [41] Chen D, Hong W, Zhou X. Transformer network for remaining useful life prediction of lithium-ion batteries. *Ieee Access* 2022;10:19621–8.
- [42] Huang J, Liu L, Zhao H, Li T, Li B. RUL-Mamba: Mamba-based remaining useful life prediction for lithium-ion batteries. *J Energy Storage* 2025;120:116376.
- [43] Liao Y, Zhang L, Liu C. Uncertainty prediction of remaining useful life using long short-term memory network based on bootstrap method. In: 2018 IEEE international conference on prognostics and health management. ICPHM, 2018, p. 1–8.
- [44] Tang J, Xiao L. The improvement of remaining useful life prediction for aero-engines by classification and deep learning. In: 2020 11th international conference on prognostics and system health management (PHM-2020 jinan). 2020, p. 130–6.
- [45] Chen Z, Wu M, Zhao R, Guretno F, Yan R, Li X. Machine remaining useful life prediction via an attention-based deep learning approach. *IEEE Trans Ind Electron* 2020;68(3):2521–31.
- [46] Liu H, Liu Z, Jia W, Lin X. Remaining useful life prediction using a novel feature-attention-based end-to-end approach. *IEEE Trans Ind Inform* 2020;17(2):1197–207.
- [47] Falcon A, D'Agostino G, Lanz O, Brajnik G, Tasso C, Serra G. Neural Turing machines for the remaining useful life estimation problem. *Comput Ind* 2022;143:103762.
- [48] Mo H, Iacca G. Evolutionary neural architecture search on transformers for RUL prediction. *Mater Manuf Process* 2023;38(15):1881–98.
- [49] Van der Maaten L, Hinton G. Visualizing data using t-SNE. *J Mach Learn Res* 2008;9(11).